

Foundations of Machine Learning

Salvador Ruiz Correa

September 26, 2026

This handout starts with a short, example-driven introduction to what machine learning actually is, then walks through four foundations that underlie almost every method in the field: *function approximation*, *optimization*, *generalization*, and *representation learning*. Each section is layered: a plain-language opening for newcomers, a mathematical middle for students ready for formalism, and an advanced remark for those who want a glimpse of current research. You can read only the “First look” paragraphs for a conceptual overview, or work through every layer for a fuller picture.

What Is Machine Learning?

First look. Teaching a computer by example instead of by instruction. In traditional programming, a person writes explicit rules: *if the email contains “free money,” mark it as spam*. Machine learning flips this around. Instead of writing the rules, you give the computer many examples of inputs and the outputs you want, and it works out the rule on its own. Three everyday cases illustrate the idea:

- **Spam filtering:** shown thousands of emails labeled “spam” or “not spam,” the system learns which patterns of words and senders tend to go with each label.
- **Movie recommendations:** shown what millions of people watched and enjoyed, a system learns to predict what a given viewer might like next.
- **Photo tagging:** shown many photos labeled “cat,” “dog,” or “car,” a system learns to recognize the same objects in new photos it has never seen.

In each case the ingredients are the same: a large set of examples, a question we want answered about new, unseen cases, and a computer program that improves its answers as it sees more examples.

Going deeper. Four broad flavors of learning. Most machine learning problems fall into one of a few families:

- **Supervised learning:** every example comes with the correct answer attached (spam / not spam; house price; object label). The goal is to predict the answer for new, unlabeled examples. This is the setting used throughout the rest of this handout.

- **Unsupervised learning:** examples have no attached answer; the goal is to find structure on their own, such as grouping similar customers together or compressing images into a smaller representation.
- **Self-supervised learning:** labels are manufactured automatically from the data itself — for example, hiding a word in a sentence and asking the model to predict it, or hiding part of an image. No human ever writes a label, yet the model ends up with the kind of general-purpose representations described in Section . This is the paradigm behind large language models and modern vision backbones.
- **Reinforcement learning:** an agent takes actions in an environment (e.g., moves in a game) and learns from rewards and penalties which actions work best over time.

Deep learning is not a fifth flavor sitting beside these; it is a subarea of machine learning defined by *how* the model is built: compositions of many simple layers, as in the neural networks introduced rather than by which of the four learning problems above is being solved. A neural network can be trained in a supervised, unsupervised, self-supervised, or reinforcement-learning setting; “deep” just means the model itself is a deep composition of layers.

Going deeper. Key tools, by learning type. A short, practical map of widely used software for each family:

- **Supervised learning:** scikit-learn, XGBoost, LightGBM, Keras / TensorFlow, PyTorch.
- **Unsupervised learning:** scikit-learn (*k*-means, PCA, DBSCAN), UMAP, HDBSCAN.
- **Self-supervised learning:** Hugging Face Transformers (BERT, GPT-style models), SimCLR and MoCo (vision), fairseq.
- **Reinforcement learning:** OpenAI Gymnasium, Stable-Baselines3, RLlib.

For the curious. Why these four pillars? Whichever flavor of learning is in play, the same four questions decide whether it will work in practice: *Can our model represent the pattern we care about?* (function approximation), *can we find good settings for it?* (optimization), *will it work on data it has not seen?* (generalization), and *what inputs should it actually look at?* (representation learning). The next four sections take these questions one at a time, returning throughout to concrete examples — spam filtering, house-price prediction, and image recognition — so the mathematics stays anchored to something tangible.

Takeaway. Machine learning replaces hand-written rules with rules learned from labeled examples. The rest of this handout asks: what makes that learning process actually work?

Function Approximation

First look. Machine learning is about guessing a rule from examples. Suppose you want a program that decides whether an email is spam. You cannot write down the exact rule by hand—there are too many ways spam can look. Instead, you show the program thousands of emails labeled “spam” or “not spam,” and it learns a rule from them. That rule is a *function*: it takes an email as input and produces a label as output. The true rule—the one that would label every possible email correctly—is hidden from us. We only see examples it produced. Learning means building a stand-in for that hidden rule.

A second example makes the same idea concrete in a different setting: predicting the sale price of a house from its size, location, and age. Nobody can write an exact formula for “price,” but given thousands of past sales we can learn a function that maps house features to a predicted price. The spam filter and the house-price predictor are the same kind of problem in disguise: guess an unknown function from labeled examples.

Going deeper. Formally, there is an unknown target function

$$f^* : \mathcal{X} \rightarrow \mathcal{Y}$$

mapping inputs $x \in \mathcal{X}$ (emails, images, patient records) to outputs $y \in \mathcal{Y}$ (labels, probabilities, real numbers). We never observe f^* directly; we observe samples (x_i, y_i) with $y_i \approx f^*(x_i)$, possibly corrupted by noise. We choose a *hypothesis class* $\mathcal{H}$ —a family of candidate functions—and search within it for a function $h \in \mathcal{H}$ that matches the data.

Common hypothesis classes a beginner should recognize:

- **Linear models:** $h(x) = w^\top x + b$. Simple, interpretable, the basis of linear regression and linear SVMs.
- **Polynomials:** $h(x) = \sum_{k=0}^p a_k x^k$. A classical way to capture curvature.
- **Decision trees:** piecewise-constant functions defined by splits on features. Used in CART, random forests, and gradient boosting.
- **Kernel machines:** $h(x) = \sum_i \alpha_i k(x_i, x)$ for a kernel k (e.g., Gaussian). The idea behind kernel SVMs and Gaussian processes.

- **Neural networks:** compositions $f_\theta(x) = W_L \sigma(W_{L-1} \cdots \sigma(W_1 x))$, where σ is a nonlinearity.

For the curious. Why the choice of $\mathcal{H}$ matters. Two errors hide inside “approximate f^* ”:

1. **Approximation error:** even the best $h \in \mathcal{H}$ may differ from f^* . A linear model cannot represent a spiral boundary.
2. **Estimation error:** with finitely many samples we cannot identify the best $h \in \mathcal{H}$ exactly.

Richer $\mathcal{H}$ reduces approximation error but increases estimation error—the *bias–variance trade-off*. The universal approximation theorem says a sufficiently wide network can approximate any continuous function on a compact domain,¹ but it says nothing about whether we can *find* the parameters or *generalize* from finite data. That is why the next two sections matter.

¹ Cybenko (1989); Hornik et al. (1989).

Takeaway. Function approximation asks: *can* our model represent the relationship we care about?

Optimization: Finding Good Parameters

First look. Learning is a search for the best settings. Once you have chosen a family of candidate rules, you need a way to pick the best one. Imagine tuning a radio: you turn a dial, listen, and adjust until the signal is clear. In machine learning, the “dial” is a set of numbers called *parameters*, and the “signal quality” is measured by a *loss function* that says how wrong the model is. Optimization is the process of turning the dials to make the loss as small as possible. For the house-price example from Section , the dials are the weights w and b in $h(x) = w^\top x + b$, and the loss might simply be how far off each predicted price is from the actual sale price.

Going deeper. The empirical risk. Given training data $\{(x_i, y_i)\}_{i=1}^n$ and a loss $\ell(\hat{y}, y)$ (squared error for regression, cross-entropy for classification), we define

$$\hat{\mathcal{R}}_n(h) = \frac{1}{n} \sum_{i=1}^n \ell(h(x_i), y_i),$$

and we search for

$$h^* = \arg \min_{h \in \mathcal{H}} \hat{\mathcal{R}}_n(h).$$

For parametric models $h = f_\theta$ this becomes

$$\theta^* = \arg \min_{\theta \in \mathbb{R}^d} \hat{\mathcal{R}}_n(\theta).$$

Going deeper. Gradient descent. If $\hat{\mathcal{R}}_n$ is differentiable, the gradient $\nabla_{\theta} \hat{\mathcal{R}}_n$ points in the direction of steepest increase. So we step the other way:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \hat{\mathcal{R}}_n(\theta_t),$$

where $\eta > 0$ is the *learning rate*. Too small and progress is slow; too large and we may oscillate or diverge.

Going deeper. Stochastic gradient descent. For large n , computing the full gradient is expensive. SGD uses a mini-batch $\mathcal{B} \subset \{1, \dots, n\}$:

$$\theta_{t+1} = \theta_t - \eta \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\theta} \ell(f_{\theta}(x_i), y_i).$$

The gradient estimate is unbiased: $\mathbb{E}[\nabla_{\mathcal{B}} \hat{\mathcal{R}}_n] = \nabla_{\theta} \hat{\mathcal{R}}_n(\theta)$ when the batch is sampled uniformly.

For the curious. Convex versus non-convex. For linear and logistic regression, $\hat{\mathcal{R}}_n(\theta)$ is convex: every local minimum is global, and we have reliable algorithms with convergence guarantees. For neural networks, $\hat{\mathcal{R}}_n(\theta)$ is non-convex, with many local minima, saddle points, and flat regions. Yet gradient methods work remarkably well in practice. Why?

- **Noise helps.** SGD's gradient noise helps escape saddle points and sharp minima.²
- **Implicit regularization.** SGD tends to find simple, well-generalizing solutions even without explicit penalties.³
- **Practical variants.** Momentum, Nesterov acceleration, AdaGrad, RMSProp, and Adam adapt the step size per parameter.⁴

Takeaway. Optimization asks: *can we find* a good function in $\mathcal{H}$ efficiently?

² Ge et al. (2015); Keskar et al. (2017).

³ Soudry et al. (2018).

⁴ Adam (Kingma & Ba, 2015) is the default in many libraries.

Generalization: Why Training Success Is Not Enough

First look. Good on the training set does not mean good in the real world. A student who memorizes past exam answers may score perfectly on those exact questions and fail on new ones. A model that memorizes the training data is the same: it looks perfect during training but fails on new examples. *Generalization* is the ability to perform well on data the model has never seen. It is the difference between memorizing and understanding. A spam filter that only recognizes the exact emails it was trained on is useless; a house-price model that only reproduces the sale prices already in its training set tells us nothing about a house that just came on the market.

Going deeper. The generalization gap. Let

$$\mathcal{R}(h) = \mathbb{E}_{(x,y) \sim p_{\text{data}}}[\ell(h(x), y)]$$

be the *true risk* (performance on the whole population) and $\hat{\mathcal{R}}_n(h)$ the empirical risk (performance on the training sample). The *generalization gap* is $\mathcal{R}(h) - \hat{\mathcal{R}}_n(h)$. We want it small for the hypothesis we deploy.

Going deeper. Capacity measures. Classical learning theory bounds the gap using measures of the “richness” of $\mathcal{H}$:

- **VC dimension:** the largest number of points $\mathcal{H}$ can shatter (label in all 2^m possible ways). A linear classifier in $\mathbb{R}^d$ has VC dimension $d + 1$.
- **Rademacher complexity:** how well $\mathcal{H}$ can fit random noise; tighter and data-dependent.
- **PAC learning:** “probably approximately correct”—with enough samples, with high probability, the learned hypothesis is close to the best in $\mathcal{H}$.

Going deeper. Regularization. In practice we control capacity with penalties and constraints: ridge (ℓ_2), lasso (ℓ_1), dropout, weight decay, early stopping, and data augmentation.

For the curious. The deep learning surprise. Deep networks often have *more parameters than training points* and can fit random labels perfectly, yet they generalize well on real data. Naive capacity bounds fail to explain this. Current explanations include:

- **Norm-based bounds:** generalization depends on the product of weight matrix norms, not just parameter count.
- **Algorithmic stability:** SGD itself provides stability guarantees.⁵
- **Double descent:** as capacity grows past the interpolation threshold, test error can decrease again.⁶
- **Implicit regularization:** the optimizer, not just the model class, determines which solution is found.

⁵ Bousquet & Elisseeff (2002).

⁶ Belkin et al. (2019); Nakkiran et al. (2019).

Takeaway. Generalization asks: *will the model work on new data?*

Representation Learning: Choosing the Right Features

First look. The features you feed a model often matter more than the model itself. Imagine trying to recognize faces using only the

Concept	Key Reference
VC dimension	Vapnik & Chervonenkis (1971)
Rademacher complexity	Bartlett & Mendelson (2002)
Uniform stability	Bousquet & Elisseeff (2002)
Double descent	Belkin et al. (2019)
Implicit bias of GD	Soudry et al. (2018)

Table 1: Milestones in generalization theory.

average brightness of each image. No model, however powerful, can succeed—the features have thrown away the information that matters. Give the same model well-chosen features (eye spacing, nose shape, etc.) and even a simple classifier can work. *Representation learning* is the art and science of choosing or learning those features. In the house-price example, “square footage” and “distance to downtown” are far more useful features than, say, the street name spelled out as raw text; picking (or learning) good features is often what separates a mediocre model from an excellent one.

Going deeper. Classical feature engineering. Before deep learning, practitioners designed features by hand:

- **Vision:** SIFT, HOG, edge detectors, color histograms.
- **Language:** bag-of-words, TF-IDF, n -grams, POS tags.
- **Speech:** MFCCs, spectrograms.
- **Tabular:** domain-specific ratios, interactions, aggregations.

Kernel methods are a classical middle ground: they implicitly map inputs into a high-dimensional feature space where linear separation becomes possible.

Going deeper. Learned representations. Deep learning replaces hand-designed features with learned ones. A deep network is a composition

$$f_{\theta}(x) = g_{\theta_L} \circ g_{\theta_{L-1}} \circ \cdots \circ g_{\theta_1}(x),$$

where each layer g_{θ_ℓ} transforms its input into a new representation. Early layers capture low-level structure (edges, tones); later layers capture high-level concepts (objects, meanings).

For the curious. What makes a good representation? Several principles recur:

- **Distributed representations:** inputs encoded by patterns of activations across many units, enabling combinatorial generalization.⁷
- **Hierarchy:** deep compositions build complex features from simple ones, mirroring the structure of natural data.

⁷ Hinton (1986); Bengio et al. (2013).

- **Disentanglement:** different dimensions capture independent factors of variation (pose, lighting, identity).
- **Self-supervised learning:** pretext tasks (masked language modeling, contrastive learning) learn useful representations without labels.⁸
- **Transfer learning:** representations learned on one task often transfer to others, enabling few-shot and zero-shot learning.

⁸ BERT, GPT, SimCLR, MoCo.

Representation learning is the thread that connects the other three pillars. Better representations reduce approximation error (the function we need is simpler), improve optimization (the loss landscape is friendlier), and aid generalization (the model captures real structure rather than memorizing noise).

Takeaway. Representation learning asks: *what features should the model use?*

How the Four Pillars Fit Together

First look. The four topics are not separate subjects. Return to the spam filter, the house-price predictor, and the photo tagger from Section : each one needs a model that *can* represent the pattern (approximation), a way to *find* good settings for it (optimization), a way to make sure it *works* on new examples (generalization), and good inputs to work from (representation). These are four questions you should ask about *any* machine learning method, from linear regression to a large language model:

1. **Function approximation:** What can the model represent?
2. **Optimization:** How do we find good parameters?
3. **Generalization:** Why should it work on new data?
4. **Representation learning:** What features does it use or learn?

Going deeper. A chain of dependencies. A model that cannot represent the truth will fail no matter how well we optimize. A model we cannot optimize will never reach its potential. A model that generalizes poorly is useless in practice. A model with poor representations will need to be much larger and much better optimized to compensate.

For the curious. A unifying tension. All four pillars sit in tension between *expressive power* (can we represent the truth?), *computational tractability* (can we find it?), and *statistical reliability* (will it work on new data?). Progress in machine learning is often progress in managing this tension—new architectures, new optimizers, new theory, new representations.

For students: whenever you encounter a new method, ask the same four questions. What is its hypothesis class? How is it optimized? Why does it generalize? What representation does it use or learn? These questions will organize your understanding of the entire field, from a linear regression to a transformer.